



VeBox Support Agent

Sistema inteligente de análisis y gestión de tickets de soporte

python 3.12+ uv package manager OpenAI Compatible API BERT embeddings Chart.js visualizations

Qué es

VeBox Support Agent es una suite de herramientas diseñada específicamente para el equipo de **HelpDesk del departamento VeBox**. El sistema automatiza todo el ciclo de vida del análisis de tickets de soporte: desde la extracción automática de datos de GLPI, pasando por la clasificación inteligente mediante agentes de IA, hasta la generación de reportes visuales con métricas accionables.

El corazón del sistema es un **framework propio de orquestación de agentes** que permite construir pipelines de IA modulares, extensibles y fáciles de mantener. Este framework proporciona abstracciones para agentes, herramientas, pipelines y comandos CLI, todo con tipado estático y diseño async-first.

Características principales

- **Scraping inteligente** — Extracción automática de tickets desde GLPI con filtros por fecha y keywords.
- **Clasificación con IA** — Sistema de 3 agentes que clasifica, audita y propone mejoras a la taxonomía.
- **Análisis conversacional** — Agente que responde preguntas en lenguaje natural sobre los datos.
- **Reportes interactivos** — Dashboards HTML con 30+ métricas y visualizaciones con Chart.js.
- **Framework extensible** — Arquitectura modular para añadir nuevos agentes y herramientas.

Qué problema resuelve

El equipo de soporte de VeBox gestiona miles de tickets al año a través de GLPI. Sin herramientas adecuadas, el análisis de estos datos es manual, inconsistente y consume tiempo valioso que podría dedicarse a resolver incidencias.

Problema	Impacto	Solución
 Extracción manual	Horas perdidas exportando y limpiando datos.	Scraping automático con un solo comando.
 Clasificación inconsistente	Categorías mal asignadas, métricas poco fiables.	IA que clasifica según taxonomía definida por el equipo.
 Falta de visibilidad	Sin dashboards, sin KPIs, decisiones a ciegas.	Reportes HTML con métricas de rendimiento en tiempo real.
 Análisis superficial	Solo conteos básicos, sin insights profundos.	Agente de IA que detecta patrones y genera recomendaciones.
 Taxonomía estática	Categorías obsoletas que no evolucionan.	Sistema que propone mejoras basadas en datos reales.



Arquitectura

El sistema sigue una arquitectura de **pipeline de datos** donde cada módulo procesa y enriquece la información para el siguiente. Todo está orquestado por un framework propio que gestiona agentes, herramientas y comandos.



Módulos del sistema

Módulo	Descripción	Tecnología
 Scraper	Extrae tickets de GLPI mediante web scraping. Soporta rangos de fechas flexibles (horas, días, semanas, meses) y modo automático que detecta desde cuándo scrapear. Filtra por keywords relevantes para VeBox.	httpx, BeautifulSoup

 Clasificador	Pipeline de 3 agentes IA que trabajan en secuencia: ClassifierAgent asigna categorías, AuditorAgent evalúa la calidad, y TaxonomyEditorAgent propone mejoras. La taxonomía nunca se modifica automáticamente.	LLM (OpenAI-compatible)
 Analyst	Agente conversacional que responde preguntas en lenguaje natural. Tiene acceso a 30+ herramientas de análisis y genera hallazgos clasificados por severidad con recomendaciones priorizadas.	LLM + Tools
 Reporter	Genera informes HTML autocontenidos con visualizaciones interactivas. Incluye dashboard ejecutivo, métricas de SLA, rendimiento por técnico, patrones temporales y estado actual del backlog.	Chart.js

Configuración

1. Instalación

El proyecto usa **uv** como gestor de paquetes por su velocidad y reproducibilidad:

```
# Clonar el repositorio
git clone <repo-url>
cd support-agent

# Instalar dependencias (uv las resuelve en segundos)
uv sync

# Crear archivo de configuración
cp .env.example .env
```

2. Variables de entorno

Edita el archivo **.env** con tus credenciales:

```
#  API de IA (requerido para clasificación y análisis)
BITBOX_API_KEY=sk-your-key-here
BITBOX_BASE_URL=https://ia.bitboxwebs.com/api/v1
BITBOX_MODEL=bb-agent:qwen
BITBOX_TEMPERATURE=0.1
```

```
# 🌐 Cookie de GLPI (requerido para scraping)
GLPI_COOKIE=your-session-cookie
```

3. Obtener credenciales

Credencial	Dónde obtenerla	Notas
API Key	ia.bitboxwebs.com → Settings → Account → API Key.	Necesaria para los agentes de IA.
Cookie GLPI	DevTools (F12) → Network → Buscar cookie <code>glpi_ca9cd5b1eb702912f0bb6bc7a8ba35dd</code> .	Expira periódicamente, renovar si falla el scraping.

⚡ Quick Start

Una vez configurado, puedes empezar a usar el sistema con estos comandos:

```
# 1 Extraer tickets de GLPI (última semana)
uv run python scripts/scrape_tickets.py --date-range 7DAY

# 2 Normalizar los tickets extraídos
uv run python scripts/normalize_tickets.py

# 3 Generar un reporte visual del estado actual
uv run python -m src.cli report

# 4 Hacer preguntas al agente de análisis
uv run python -m src.cli analyze --query "¿Cuál es el estado general del helpdesk?"
uv run python -m src.cli analyze --query "¿Qué categorías tienen más reaperturas?"

# 5 Clasificar tickets con el pipeline de IA
uv run python -m src.cli pipeline --limit 100
```

El reporte HTML se abre automáticamente en el navegador y se guarda en `outputs/report.html`.

📄 Documentación

- 🏗️ ARQUITECTURA
- ⚙️ FRAMEWORK
- 🔍 SCRAPER
- 📄 CLASIFICADOR
- 📊 ANÁLISIS

Comandos

Scraping de tickets

Extrae tickets directamente desde GLPI. El modo **auto** detecta automáticamente desde cuándo scrapear basándose en la última ejecución.

```
# Modo automático (recomendado para ejecuciones periódicas)
uv run python scripts/scrape_tickets.py --date-range auto

# Por rango específico
uv run python scripts/scrape_tickets.py --date-range 24HOUR    # Últimas 24 horas
uv run python scripts/scrape_tickets.py --date-range 7DAY      # Última semana
uv run python scripts/scrape_tickets.py --date-range 1MONTH    # Último mes
```

Clasificación con IA

El sistema de clasificación usa 3 agentes que trabajan en pipeline. Puedes ajustar el tamaño de batch y la frecuencia de auditoría.

```
# Solo clasificar (sin auditoría)
uv run python -m src.cli classify --limit 100

# Pipeline completo: clasificar → auditar → proponer cambios
uv run python -m src.cli pipeline --limit 500

# Con auditoría cada 5 batches (mejor detección de patrones)
uv run python -m src.cli pipeline --limit 500 --audit-every 5

# Procesamiento concurrente (más rápido)
uv run python -m src.cli pipeline --limit 500 --concurrent 3
```

Análisis y reportes

El agente de análisis responde preguntas en lenguaje natural. El generador de reportes crea dashboards HTML interactivos.

```
# Preguntas al agente
uv run python -m src.cli analyze --query "¿Cuál es el estado del helpdesk?"
uv run python -m src.cli analyze --query "¿Qué técnicos tienen mejor rendimiento?"
uv run python -m src.cli analyze --query "Identifica cuellos de botella" --verbose

# Generar reporte HTML
uv run python -m src.cli report                                # Todos los datos
```

```
uv run python -m src.cli report --year 2024      # Filtrar por año
uv run python -m src.cli report --last-n-days 30  # Últimos 30 días
```

Estructura del Proyecto

support-agent/

src/

core/

 Framework de orquestación

└─ BaseAgent: clase base para agentes

└─ Registry: decoradores @agent, @tool,

@command

└─ Tool: sistema de herramientas para LLM

└─ Pipeline: orquestación de agentes

└─ MCP: Model Context Protocol

agents/

 Agentes de IA

└─ AnalystAgent: análisis conversacional

└─ ClassifierAgent: clasificación de tickets

└─ AuditorAgent: auditoría de clasificaciones

└─ TaxonomyEditorAgent: propuestas de mejora

tools/

 Herramientas de análisis

└─ analysis/

└─ 30+ funciones: SLA, tiempos, volumen,

etc.

cli.py

 Punto de entrada CLI

scripts/

 Scraper de GLPI

└─ scrape_tickets.py

└─ discover_tickets.py

 Descubrimiento de IDs

└─ normalize_tickets.py

 Normalización de texto

data/

 Datos de entrada

└─ raw/

└─ taxonomy.json

└─ Taxonomía de categorías

└─ tickets_clean.jsonl

└─ Tickets normalizados

outputs/

 Resultados generados

└─ classifications/

└─ Clasificaciones por batch

└─ reports/

└─ Reportes de auditoría

└─ report.html

└─ Dashboard HTML

docs/

 Documentación detallada

Roadmap

El proyecto está en desarrollo activo. Estas son las funcionalidades actuales y las planificadas:

Estado	Feature	Descripción
✓	Scraper GLPI	Extracción automática con rangos flexibles y modo auto.
✓	Sistema de clasificación	Pipeline de 3 agentes con auditoría y propuestas.
✓	Reportes HTML	Dashboard con 30+ métricas y visualizaciones interactivas.
✓	Agente de análisis	Queries en lenguaje natural con hallazgos y recomendaciones.
✓	Framework de agentes	Core propio con Registry, Tools, Pipelines y MCP.
❑	Sistema RAG	Búsqueda semántica para asistir en resolución de tickets.
❑	Dashboard real-time	Interfaz web con actualización en tiempo real.
❑	Alertas SLA	Notificaciones automáticas cuando un ticket está en riesgo.

Stack Tecnológico

 PYTHON

3.12+

UV

PACKAGE MANAGER

BERT

EMBEDDINGS

OPENAI

COMPATIBLE API

 CHART.JS

VISUALIZATIONS

HTTPX

ASYNC HTTP

Departamento VeBox · HelpDesk Analytics

[Empezar](#) · [Arquitectura](#) · [Framework](#) · [Análisis](#)